Experiment No:18
Experiment to understand KNN algorithm for a given data set

AIM:
To understand KNN algorithm for a given dataset

DESCRIPTION:
Understand the KNN algorithm for the dataset given.

CODE:

```
import numpy as np
import pandas as pd
df=pd.read_csv('/content/Iris.csv')
df.info()
df.Species.value_counts()
df.head()
features=df.iloc[:,1:-1].values
label=df.iloc[:,5].values
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
xtrain,xtest,ytrain,ytest=train_test_split(features,label,test
_size=.2,random_state=42)
model_KNN=KNeighborsClassifier(n_neighbors=5)
model_KNN.fit(xtrain,ytrain)
print(model_KNN.score(xtrain,ytrain))
print(model_KNN.score(xtest,ytest))
from sklearn.metrics import confusion_matrix
print(confusion_matrix(ytest,model_KNN.predict(xtest)))
from sklearn.metrics import classification_report
print(classification_report(ytest,model_KNN.predict(xtest)))
```

OUTPUT:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Id             150 non-null    int64
 1   SepalLengthCm  150 non-null    float64
 2   SepalWidthCm   150 non-null    float64
 3   PetalLengthCm  150 non-null    float64
 4   PetalWidthCm   150 non-null    float64
 5   Species        150 non-null    object
dtypes: float64(4), int64(1), object(1)
memory usage: 7.2+ KB
0.9666666666666667
1.0
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
                precision    recall  f1-score   support

    Iris-setosa       1.00      1.00      1.00        10
Iris-versicolor       1.00      1.00      1.00         9
 Iris-virginica       1.00      1.00      1.00        11

       accuracy                           1.00        30
      macro avg       1.00      1.00      1.00        30
   weighted avg       1.00      1.00      1.00        30
```

RESULT:

Thus, the K-Nearest Neighbors (KNN) algorithm was successfully implemented on the Iris dataset to classify iris flowers into three species: Iris-setosa, Iris-versicolor, and Iris-virginica. The model was trained using 80% of the dataset and tested using the remaining 20%. It achieved 100% test accuracy, and the confusion matrix and classification report were generated successfully.